Cloud Computing

Exercise - 4 Total points: 10

---

1. Get the system information of your machine using the command prompt/terminal. Discuss the commands used and describe the output in detail. The information of interest for you is no of cores, RAM, clock speed of the CPU, current memory usage, the processes running etc. which are relevant for multiprocessing/multithreading. It is expected that you have tried several (probably 5-10) possible commands. (1 point)
   Note: The output screenshots should prove that the respective student has performed the activities.

2. Comparing single-threaded and multithreaded **Java** programs to download online books

   a. Write a simple single-threaded Java program to download and save books from the Gutenberg repository, which has open-licensed and free books for everyone to use. For example, use the following link to list a set of random books from the Gutenberg repository: https://www.gutenberg.org/ebooks/search/?sort_order=random
   b. Create a function download_book(gutenberg_id) that receives the Gutenberg book number (ID) as an argument, downloads the corresponding book (as a text file), and saves it as a separate file on the computer.
      For example, if the gutenberg_id parameter is 45774, then the address of the book in UTF8 text format would be:
      https://www.gutenberg.org/cache/epub/45774/pg45774.txt
      https://www.gutenberg.org/cache/epub/<gutenberg_id>/pg<gutenberg_id>.txt
   c. In the main method of the program, create a list of book ids and loop over the list that calls the *download_book* function to download each of the books identified by *gutenberg_id*. (1 point)
   d. Have a provision to download a range of book ids. E.g. 45524 to 45573.

   e. Now measure the time taken by the program to download 50 books.
      i. You can use System.currentTimeMillis()
   f. Now improve the solution of the previous steps by executing all download_book() function calls in a separate thread, with the aim to speed up the book downloading.
   g. Use logging to check and debug threads. Use java.util.logging.Logger;
   h. Arguments to the function to be called are provided separately through the Thread. (1 point)
   i. Now measure the time taken by the threaded program to download 50 books.
   j. How much faster or slower is the book downloading using threads compared to the previous program (without threads)? (1 point)
   k. Script execution time can also be measured outside the script. In Linux, you can use the time command.
      How much faster or slower is the book downloading using threads compared to the previous program (without threads), by following the script running approach? (1 point)

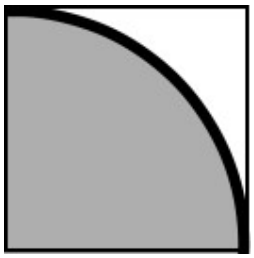
   l. If we want to download hundreds of files, it is unreasonable to start an individual thread for all of them at the same time. Instead, it is often wise to limit the maximum number of threads using thread pools. Use java.util.concurrent.Executors

and java.util.concurrent.ExecutorService and improve the multithreaded program, that uses a thread pool of size 10 threads.
Measure how long it takes to download 50 books using *thread pool* and compare the result with the previous two solutions (single threaded and multithreaded). Show the times using both the System.currentTimeMillis() and time command solutions. (1 point)

Note: Store all the intermediate java files, while sending them for evaluation. In this case, you have multiple java classes, one each for single threaded program, multithreaded program and multithreaded program with thread pool.

3. In the task 2, our threads were completely independent, and their work did not affect each other in any way. In this task, you will make threads to work together to compute a global value.
    a. Create a method download_the_book_and_search_word(gutenberg_id, word) similar to the previous download_book() method, but it searches for specific terms and counts how many times the search term occurs in the file. For example, "India" or "language" or "thread". You should loop through the contents of the book line by line.
    P.S. Books no longer need to be stored in files.
    b. The global variable SearchCount will keep the count of the search word across all the threads/books.
    c. The function must update the value of the global counter each time a match is found. Not just once at the end of thread work. (1 point)
    d. Now try to search the keyword in 50 files, using a thread pool of size 10.
    e. At the end of the program, the search word should be printed and how many times it was found in the searched books. Also print the gutenberg_id s of the books in some compact form. (1 point)

4. ***Monte Carlo PI :*** It is possible to get an estimate of the mathematical constant π by using a random process. The idea is based on the fact that the area of a circle of radius 1 is equal to π, and the area of a quarter of that circle is π/4. Here is a picture of a quarter of a circle of radius 1, inside a 1-by-1 square:



The area of the whole square is one, while the area of the part inside the circle is π/4. If we choose a point in the square at random, the probability that it is inside the circle is π/4. If we choose N points in the square at random, and if C of them are inside the circle, we expect the fraction C/N of points that fall inside the circle to be about π/4. That is, we expect 4*C/N to be close to π. If N is large, we can expect 4*C/N to be a good estimate for π, and as N gets larger and larger, the estimate is likely to improve.

We can pick a random point in the square by choosing numbers x and y in the range 0 to 1 (using Math.random()). Since the equation of the circle is x*x+y*y=1, the point lies inside the circle if x*x+y*y is less than 1. One trial consists of picking x and y and testing whether

x*x+y*y is less than 1. To get an estimate for π, you have to do many trials, count the trials, and count the number of trials in which x*x+y*y is less than 1.

Write a multithreaded program in Java that calculates the value of π. (2 points)

a. Let the user specify the number of threads
b. Let each thread generate large number of random points. E.g. 10,000. Again, let the user specify the number.
c. Take care of the synchronization of all the threads while calculating π.

Follow these guidelines while submitting the exercises

1. Generate a PDF file with answers/screenshots with name being your studentid_EX4
2. Export the eclipse project (for java code) as a zip file
3. Put a pdf file with the answers for the questions
    a. For programming tasks show screenshots of the program output for the correct and wrong inputs
    b. For commands tried, provide the screenshots
4. The programs should guide the user in putting the correct input
    a. Have detailed comments of your logic
5. Do not copy code/answers from internet. If you copy, please leave a comment with the URL on that java file
6. Plagiarism will not be tolerated and dealt through the formal channels of the university